

57. Une « struct » : ce qui va ensemble

Une troupe d'ennemis, et non six tableaux parallèles.

1. Ce qu'il faut comprendre

Un ennemi a une position, une vitesse, une santé. Sans `struct`, cela fait trois tableaux qu'il faut penser à garder alignés — et le jour où l'on en trie un sans trier les autres, les vies changent de propriétaire.

Une `struct` range ces champs ensemble. `Etoile etoiles[4];` réserve quatre enregistrements ; `etoiles[i].x` en désigne un champ. Le compilateur calcule le décalage lui-même, et `sizeof(Etoile)` dit ce qu'un enregistrement pèse.

Le numéro d'un lutin peut être **calculé** : `sprite(i, ...)` dans une boucle affiche toute la troupe, chacun avec ses valeurs à lui. C'est ce qui rend la `struct` utile plutôt que décorative.

Une `struct` peut en contenir une autre : `struct Ennemi { Point ou; uint8_t vie; };`, et l'on écrit `troupe[i].ou.x`. Le compilateur additionne les décalages lui-même.

Ce qu'elle ne sait pas faire : s'affecter d'un bloc, se passer en argument, se rendre par une fonction. On nomme le champ qu'on veut lire ou écrire — et le compilateur le dit plutôt que de traduire à peu près.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1690 octets** de cartouche.

1. UN DESSIN — CORPS

```
Tuile CORPS = {  
    "00333300",  
    "03222230",  
    "32222223",  
    "32122123",  
    "32222223",  
    "32211223",  
    "03222230",  
    "00333300",  
};
```



à l'écran

2. UN ENREGISTREMENT — ETOILE

```
struct Etoile {  
    uint8_t x;  
    uint8_t y;  
    uint8_t vitesse;  
};
```

3. UNE CONSTANTE — COMBIEN

```
const uint8_t COMBIEN = 4;
```

4. UN TABLEAU — ETOILES

```
Etoile etoiles[COMBIEN];
```

5. UNE FONCTION — PLACER()

```
void placer() {
    for (uint8_t i = 0; i < COMBIEN; i++) {
        etoiles[i].x = 20 + i * 30;
        etoiles[i].y = 32 + i * 24;
        etoiles[i].vitesse = 1 + i;
    }
}
```

6. UNE FONCTION — AVANCERTOUTES()

```
void avancerToutes() {
    for (uint8_t i = 0; i < COMBIEN; i++) {
        etoiles[i].x += etoiles[i].vitesse;

        if (etoiles[i].x > 152) etoiles[i].x = 0;

        sprite(i, etoiles[i].x, etoiles[i].y, CORPS);
    }
}
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(2, 1, "UNE TROUPE");

    placer();

    while (true) {
        image();
        avancerToutes();
    }

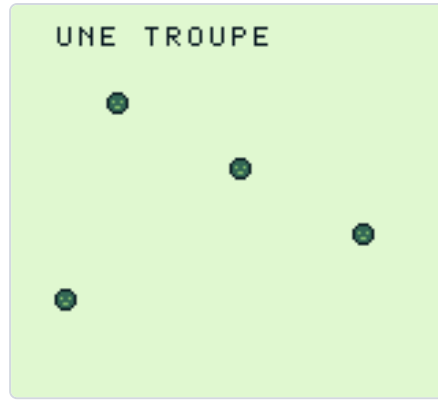
    return 0;
}
```

3. Ce qui se passe, étape par étape

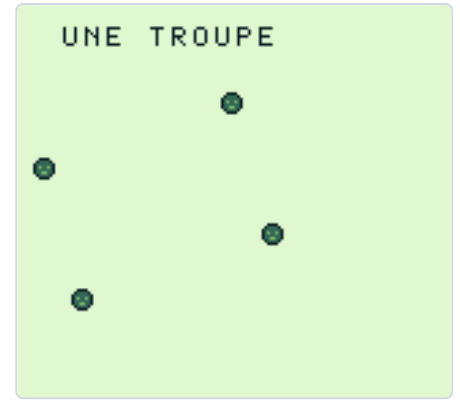
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



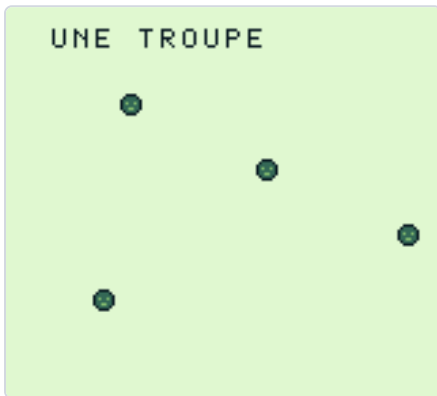
la toute première image — la console vient de démarrer — image 1



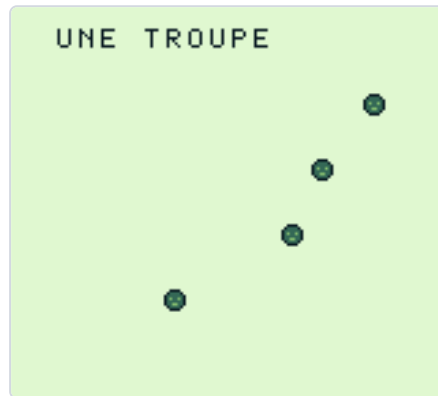
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Quatre boules qui traversent l'écran, chacune à sa vitesse.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les quatre lutins portent la tuile dessinée
- ✓ ils sont sur quatre lignes différentes
- ✓ tous avancent (30, 60, 191, 120)
- ✓ et chacun à sa vitesse (30, 60, 191, 120)
- ✓ le titre est écrit

Ensuite : **58. Le décor qui défile** — niveau 8 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++